

RAILFUSE

Opportunity-Aware Adaptive Block Planning

Smart India Hackathon 2026 | PS ID: SIH26027 | Team: Runtime Rebels

25

Tasks

10

Blocks

34

Trains

103

Tests

1 click

Replan

3-MINUTE PITCH SCRIPT + JUDGE Q&A

All data is synthetic. Not connected to live Indian Railways systems.

0:00 - 0:40

Indian Railways runs the world's largest rail network under one management - over 13,000 trains a day, 68,000 km of track.

Maintenance teams must work inside "Engineering Blocks" - short windows of 2-4 hours at night when a section of track is cleared of trains.

Today, that scheduling is done manually:

- A maintenance manager looks at a spreadsheet, assigning tasks one by one.
- No real-time visibility into which tasks are overdue or about to expire.
- No awareness of which tasks from two departments could share the same block.

The result:

- High-debt tasks slip through the cracks and become safety risks.
- Blocks run at 40-50% efficiency when they could reach 80%+.
- Every extra possession taken = one more disruption to passengers.

*"Can we bring intelligent, explainable automation to this process
- without replacing the human decision-maker?"*

PART 2 - OUR SOLUTION

0:40 - 1:45

We built RAILFUSE - Opportunity-Aware Adaptive Block Planning.

The core idea: every maintenance block is not just a time window - it is an OPPORTUNITY to fuse multiple tasks from multiple departments into one possession.

RAILFUSE runs a 5-step pipeline every time:

- 1. MAINTENANCE DEBT SCORING:** Score every task by days overdue, times deferred, and safety-criticality. Tells us WHAT needs to happen most urgently.
- 2. FLEXIBILITY SCORING:** Calculate how many future blocks each task can still fit into. A task with only one future option gets a protection flag. Tells us HOW MUCH TIME is left.
- 3. CONFLICT DETECTION:** Before assigning anything: Is there a train in this section within a 10-minute safety buffer? Are required resources available? Does the task duration fit the block's remaining capacity?
- 4. MULTI-DEPT OPPORTUNITY FUSION:** [OUR CORE INNOVATION] Build a compatibility graph of which tasks from different departments can share a block safely. Fuse compatible tasks. One possession. Two departments. Zero extra disruption.
- 5. FULL EXPLAINABILITY:** Every task gets a plain-language reason. "SELECTED for BLK003: debt 28.5, fused with T004 (S&T). DEFERRED: 3 future windows available." No black box. Every decision can be audited.

1:45 - 2:30

Existing systems fall into two categories:

CATEGORY A - Manual Scheduling Tools (Spreadsheets, basic IRMS modules)

- No scoring, prioritization, or conflict detection.
- Cannot combine tasks across departments.
- Cannot adapt when a new emergency task arrives.

CATEGORY B - Generic Academic / Commercial Optimizers

- Not built for Indian Railways specific constraints.
- No maintenance debt or flexibility score concepts.
- No multi-department fusion. No explainability.

WHAT WE DO DIFFERENTLY	WHY IT MATTERS
Maintenance Debt formula	<i>Quantifies urgency - no guessing</i>
Flexibility Score + Protection	<i>Prevents stranded low-flex tasks</i>
Multi-Dept Compatibility Graph	<i>Maximizes each possession's value</i>
Zero-Possession Preference	<i>Reduces disruptions to passengers</i>
Real-Time Dynamic Replanning	<i>Handles emergencies without calls</i>
Per-Task Plain-Language Reasons	<i>Builds trust with field officers</i>
Weekly + Monthly Horizon Views	<i>Supports strategic capacity planning</i>
103 deterministic tests	<i>Verifiable, reproducible behavior</i>

We are not claiming to solve a problem no one has studied. Block scheduling, constraint satisfaction, and explainability are established fields. RAILFUSE combines them specifically for the Indian Railways maintenance context - debt-aware scoring, cross-department fusion, and dynamic replanning - into a single, coherent, demonstrable workflow. That integration is our contribution.

PART 4 - CLOSE

2:30 - 3:00

RAILFUSE is a prototype - all data is synthetic, not live IR data. But the workflow it demonstrates is real and implementable:

- Clean FastAPI service with 15 endpoints, fully documented.
- Deterministic, transparent, testable optimizer (seed 42, 103 tests).
- Frontend works in any browser - no installation needed.
- Deployable on a divisional server in under an hour, zero external cloud dependencies.

*"Give every maintenance officer a tool that tells them
not just WHAT to schedule - but WHY, and WHAT HAPPENS if they don't.
That is RAILFUSE."*

25	Maintenance tasks in the demo dataset
10	Engineering blocks across 4 corridors
34	Train movements tracked (Delhi + Bangalore corridors)
10 min	Safety buffer enforced around all train windows
103	Automated tests - all passing, all deterministic
Seed 42	Optimizer is fully reproducible: same input = same output
1 click	Time to inject an emergency task and get a new full plan
4	Planning horizons: Block / Daily / Weekly / Monthly
0	External paid APIs or cloud AI dependencies

LIKELY JUDGE QUESTIONS - QUICK ANSWERS

Q: Is this connected to real Indian Railways data?

A: No. All data is synthetic, purpose-built for this demonstration. The system is designed to plug into real IRMS data exports with a REST wrapper.

Q: How is the optimizer different from a simple greedy algorithm?

A: It is a multi-pass heuristic: (1) Score all tasks. (2) Check conflicts. (3) Find compatible dept combinations. (4) Apply zero-possession preference. (5) Protect low-flexibility tasks for future windows. A greedy algorithm takes the highest-debt task per block without considering cross-block impact or future stranding.

Q: Why not use a neural network or ML model?

A: Explainability is a hard requirement for safety-critical infrastructure. A neural model cannot tell a field officer WHY a task was deferred. Our rule-based heuristic with transparent scoring can. We chose correctness and trust over prediction accuracy.

Q: What would it take to deploy this at a real division?

A: (1) A data feed from IRMS/SAP for tasks and blocks. (2) A REST wrapper around that feed. (3) This backend deployed on a divisional server. The frontend runs in any browser - no client install needed.

Q: How does this compare to what Railways already uses?

A: Existing tools are record-keeping systems - they store what was done. RAILFUSE is a decision-support system - it tells you what to do next and why. That is a fundamentally different category of tool.

Q: Is the optimizer scalable to thousands of tasks?

A: The current implementation handles SIH demonstration scale. For production, the heuristic can be parallelized by corridor/division, and conflict detection can be indexed for $O(\log n)$ lookups. This is a prototype; production engineering would address scale.